

Descrição técnica

henryjose@edu.univali.br

julia.7230575@edu.univali.br

Universidade do Vale do Itajaí

Microcontroladores

A aplicação implementa um temporizador que exibe no formato de minutos e segundos (99:59). Ela foi desenvolvida em linguagem C no ambiente *STM32CubeIDE 1.17.0*, para o microcontrolador *STM32F103C8T6*. O funcionamento do temporizador depende fundamentalmente de uma combinação de técnicas de *máquina de estados* e *tabela de despacho* para controlar o fluxo de transição de modos e ações. O temporizador utiliza como base de tempo o *SysTick* nativo do *SMT32*, enquanto seu formato de exibição no display de sete segmentos e quatro dígitos implementa método de multiplexação. Para entender como o temporizador opera, deve-se consultar o manual de usuário.

Captura de tempo com o SysTick

O *SysTick* é um temporizador de 24 bits integrado ao núcleo do *ARM Cortex-M*, que é configurado automaticamente para gerar interrupções a cada 1 milissegundo, baseado em frequências de clock do sistema. A variável global *uwTick* é incrementada a cada interrupção do *Systick* e seu valor atual é retornado pelo método nativo *HAL_GetTick*. Desta forma, é possível capturar tempo e realizar operações como comparação, incremento, decremento, passagem e controle de tempo, como também técnica de *debouncing*.

A aplicação possui a MACRO *TICK_EXPIRED*, responsável por verificar a passagem de tempo com base num intervalo desejado, em milissegundos, e uma variável para comparação. Após verificar que o tempo desejado foi atingido, a MACRO atualiza a variável de comparação com o tempo atual utilizando *HAL_GetTick* e retorna verdadeiro para indicar que o tempo desejado for atingido. Desse modo, é controlar o fluxo temporal da aplicação para implementar *debouncing* e verificar a passagem de tempo de 1 segundo.

```
#define TICK_EXPIRED(lastTick, interval) \
    ((HAL_GetTick() - (lastTick) >= (interval)) ? \
    ((lastTick) = HAL_GetTick(), 1) : 0)
```

1. Exemplo de aplicação de técnica de *debouncing* com a MACRO *TICK_EXPIRED* para eliminar falsos acionamentos causados por trepidação:

```

GPIO_PinState  current_S1  =  HAL_GPIO_ReadPin(GPIOC,
GPIO_PIN_13);
    if (current_S1 == GPIO_PIN_RESET && last_S1 ==
GPIO_PIN_SET)
    {
        if (TICK_EXPIRED(button_S1_tick, 50))
        {
            pressed = BUTTON_1;
        }
    }
    last_S1 = current_S1;

```

2. Exemplo de verificação de passagem de tempo de 1 segundo (1000 milissegundos) utilizando a macro *TICK_EXPIRED*:

```

void start_timer(void)
{
    if (TICK_EXPIRED(last_timer_tick, 1000))
    {
        if (seg > 0)
        {
            seg--;
        }
        else
        {
            if (min > 0)
            {
                seg = LIM_SEG;
                min--;
            }
            else
            {
                // TIMER CHEGOU A 00:00
                HAL_GPIO_WritePin(GPIOA,      GPIO_PIN_8,
GPIO_PIN_SET); // Alarme
                // timer_done = 1;
            }
        }
    }
}

```

Técnica de multiplexação para display de sete segmentos e quatro dígitos

O temporizador conta com um display de sete segmentos e quatro dígitos para exibir o valor em tempo real dos minutos e segundos em pares de dígitos para cada. Os dígitos possuem os barramentos dos segmentos compartilhados, mas o acionamento de cada dígito é individual. Logo, ao acionar um dígito de cada vez e,

em cada acionamento, gravar um valor numérico, enviando-os para os segmentos, milhares de vezes por segundos, implementa-se a técnica da multiplexação. No código, isso é feito pelos métodos *enable_digit* e *refresh_display*. Enquanto *enable_digit* habilita cada dígito individualmente, no intervalo de tempo em que o dígito está acionado, *refresh_display* envia o valor numérico para os segmentos. O intervalo de tempo escolhido entre o acionamento de cada dígito é 2 ms.

Método enable_digit:

```
void enable_digit(uint8_t digit)
{
    GPIOB->BSRR = (GPIO_PIN_12 | GPIO_PIN_13 | GPIO_PIN_14
| GPIO_PIN_15) << 16;
    switch (digit)
    {
        case 0:
            GPIOB->BSRR = GPIO_PIN_12;
            break;
        case 1:
            GPIOB->BSRR = GPIO_PIN_13;
            break;
        case 2:
            GPIOB->BSRR = GPIO_PIN_14;
            break;
        case 3:
            GPIOB->BSRR = GPIO_PIN_15;
            break;
    }
}
```

Método refresh_display:

Observação: o método possui uma configuração especial para o caso de troca de dígitos enquanto está configurando o valor do contador, que permite que o dígito pisque se estiver selecionado para ser incrementado.

```
void refresh_display(void)
{
    if (TICK_EXPIRED(last_display_tick, 2))
    {
        if (TICK_EXPIRED(blink_tick, 250))
        {
            blink_state = !blink_state;
        }

        // Desliga segmentos para evitar ghosting
        GPIOA->BSRR = 0x00FF << 16;
    }
}
```

```

        // Se estiver no modo SET e for o dígito
selecionado, verifica o blink_state
        // if (enable_set && current_digit ==
selected_digit_idx && !blink_state) {
            if (state == STATE_SET && current_digit ==
selected_digit_idx && !blink_state)
            {
                // Não liga o dígito (fica apagado para efeito
de blink)
                GPIOB->BSRR = (GPIO_PIN_12 | GPIO_PIN_13 |
GPIO_PIN_14 | GPIO_PIN_15) << 16;
            }
            else
            {
                GPIOA->BSRR =
segments[number[current_digit]];
                enable_digit(current_digit);
            }

            current_digit = (current_digit + 1) % DIGITS;
        }
    }
}

```